

System Model

- **Assumptions and Constraints:**
 - 1) Database workload (Data Processing System):
 - Online Transaction Processing (OLTP)
 - 2) Database operations:
 - CRUD (Create Read Update Delete) operations
 - Range-based reads (Intra-shard and Inter-shard reads)
 - Concurrent reads
 - Blocking writes (Intra-shard and Inter-shard writes)
 - Atomic insertions, updates and deletion
 - 3) Failure model:
 - Crash Fault Model
 - Exclusion of Byzantine Faults, assuming that malicious sites are not present

- **Distributed Database Management System (DDMS) Architecture:**
 - 1) Database type (single site):
 - Non-Relational database (Key-Value database) -> Amazon DynamoDB, Redis, Couchbase Server, etcd
 - Non-Relational database (Wide-Column store database) -> Apache Cassandra, Apache HBase
 - Hybrid: YugabyteDB
 - 2) Database workload:
 - Online Transaction Processing (OLTP)
 - 3) Transactions/Operations:
 - CRUD (Create Read Update Delete)
 - 4) Resources:
 - Shared-nothing architecture
 - 5) Site architecture:
 - Autonomous-Homogeneous
 - 6) Coordination and cooperation:
 - Fault tolerance: Partial replication -> Single Primary Multiple Replicas
 - Failure Detection: In-built DDMS function regarding only Crash Fault Model
 - Mutual exclusion: (Token Ring Algorithm)
 - Election: In-Built DDMS
 - Consensus: Raft/Paxos (In-built DDMS function)
 - 7) Concurrency control:
 - Pessimistic approach
 - Atomicity with Two-Phase Commit (2PC)
 - 8) Consistency:
 - Internal database consistency (High partition tolerance and high availability)
 - 9) Data flow control:

- In-use DDMS
- 10) Topology of sites:
 - Ring topology
- 11) Cluster architecture:
 - Geo-based cluster consisting of a ring topology of multiple sites
- **Middleware Model/Architecture (Application-level):**
 - 1) Components to implement:
 - Query routing
 - Sharding
 - Cluster management
 - 2) Data sharding:
 - Key-based/Hashed-based sharding (Algorithmic sharding) with consistent hashing
 - 3) Shard distribution:
 - Multiple shards within each site
 - Disjoint shards within and among the sites
 - 4) Communication management:
 - Intra-shard communication within each site
 - Inter-shard communication between multiple sites
 - 5) Transactions:
 - CRUD (Create Read Update Delete) queries including cross-shard joins
- **Database Middleware Implementation:**
 - 1) Programming language:
 - Python
 - 2) Client communication:
 - Message-/Query-oriented
 - API calls to middleware
 - Separation of functionalities between client and middleware
 - Global logical database view
 - 3) Distributed Database Management System (DDMS) communication:
 - API calls between middleware and DDMS
 - 4) Functionalities:
 - 5) Components to implement:
 - Query routing
 - Shard management
 - Cluster management

Open Questions:

- 1) Single point of failure: Middleware. What happens when the middleware fails?
- 2) Transparency: Middleware needs to know all the sites and all authenticated clients.
- 3) Authentication: Middleware needs to accept queries from the authenticated clients.
- 4) Client implementation: Client needs to load middleware module for API calls.
- 5) Query routing: Middleware needs to route the query to designated sites
- 6) Performance overhead:
 - a. Middleware needs to distribute shards within each site and among all other sites in consistent manner.
 - b. Middleware needs to send requests to the DDMS for all concurrent queries, which leads to performance bottleneck and scalability problem.
 - c. Middleware needs to aggregate intermediate results from multiple shards to provide desired and correct output to clients' query. OR: Delegate the functionality to the DDMS
 - d. Middleware needs to manage geo-distributed cluster of sites.

